

Instructor Guide

AI-Assisted Game Development — the single-day program, session by session

Prepared for Panteon Games — Codedo · One trainer · 09:30 – 17:30

This guide is the trainer's runbook: what to have open, what to say, the exact prompts and commands for every demo and exercise, and how to recover when something breaks. Participant-facing detail lives in the two lab guides and in the exercise files inside the repos (coin-rush-workshop/Assets/Workshop/Ex0N_*/README.md and gem-hunter-workshop/Workshop/Ex0N_*.md) — when in doubt, the repo file is the source of truth.

BEFORE THE DAY (DO NOT SKIP)

- Both repos cloned and opened once on every participant machine; `unity status` shows "ready" on each.
- Your own machine: both projects open in two Unity instances, plus a **second clone of gem-hunter-workshop with ex06-start checked out** — this is your BUG-312 demo copy, so your main clone stays on main.
- Backups staged: the recorded screen video of the BUG-312 demo; the pre-generated gem set for anyone without Scenario access; a raw mesh set is not needed in this program.
- Printed or open: baseline metrics template (one per team lead), art approval checklist (Track B guide appendix).
- Test once, that morning: `./catchup.sh list` in gem-hunter-workshop, and `unity command eval '1+1'` against both editors.
- Room: one workshop room, projector, one machine per lab participant.

DAY AT A GLANCE

Time	Session	You need open
09:30 – 10:00	Opening (deck 01)	Deck 01 · Gem Hunter in play mode
10:00 – 11:00	Unity CLI from scratch, hands-on (deck 02)	Deck 02 · Coin Rush + terminal
11:15 – 12:30	Foreign-code techniques + BUG-312 demo (deck 03)	Deck 03 · ex06-start demo clone
13:30 – 15:00	Track A: Gem Hunter lab (deck 04)	Deck 04 · gem-hunter-workshop
15:15 – 16:45	Track B: the reskin (deck 05)	Deck 05 · Scenario + gem-hunter
16:45 – 17:30	Closing (deck 06)	Deck 06 · both participant outputs

09:30 · OPENING — RUN OF SHOW

1. Frame the day (slides 1–4)

One sentence each: you already use Claude Code daily, so today it drives Unity. State the principle ("tools do the producing, people make the final call"), walk the approval gate / code review / provenance slide, then the arc: from scratch in the morning, foreign code in the afternoon.

2. Record the baseline (slide 2)

Hand out the metrics template. Say explicitly: team leads return it filled in at the end of the day — T1 dev time, T2 art time, T3 bug time, first-pass rejection rate. In 90 days you compare against today's numbers.

3. Everyone plays Gem Hunter once (5 min)

Level 1 on their own machines: goals, boosters, the move counter. They must know the game before they read its code.

10:00 · UNITY CLI FROM SCRATCH — HANDS-ON (COIN RUSH)

Theory first, 15 minutes, deck 02 slides 2–4: why the CLI is official now, the three layers, how an agent drives the editor. Then everyone opens `Ex01_Start.unity` in coin-rush-workshop and follows you.

1. Connect and discover (your demo, 5 min)

```
$ unity status                # state: ready
$ unity command               # list every exposed command
$ unity command eval 'new UnityEngine.GameObject("Hello");'
```

Delete "Hello" again with `eval`. Point at the Hierarchy after each command — the room must see the editor react to the terminal.

2. Everyone builds the scene (10 min)

Participants run the `Ex01 README` commands themselves: the Ground cube (20×0.5×20 at `y=-0.25`), the Player capsule with kinematic Rigidbody, then interrogate it:

```
$ unity command eval 'UnityEngine.GameObject.Find("Player") != null' # True
```

3. The agent writes movement (15 min)

Everyone starts cClaude, pastes the Ex02 README prompt (PlayerController: WASD + click-to-move, clamp ± 9 , face movement direction, attach via CLI). **Coach the plan review out loud:** which files? inside Ex02_Movement only? Approve, play, verify:

```
$ unity command eval 'UnityEngine.GameObject.Find("Player").GetComponent("PlayerController") != null'
```

4. Coins — your demo only (5 min)

Run the Ex03 prompt yourself (coin prefab + ring of 8), then the payoff line — live inspection:

```
$ unity command eval 'UnityEngine.GameObject.Find("Coins").transform.childCount'
```

Collect one in play mode, run it again — one fewer. Say it explicitly: *this eval habit is how we prove bugs all afternoon.*

11:15 · FOREIGN CODE + BUG-312 — DEMO SCRIPT

Deck 03. Techniques first (11:15–11:45, slides 2–4): archaeology ("a superb guide and a confident liar" — map, then verify), the contract, checkpoints (./catchup.sh). Then the demo, on your ex06-start clone:

1. Reproduce by hand

Play a level, make a swap that does NOT match — it bounces back. Point at the move counter: it dropped anyway. Name the wrong fix ("give players more moves") and the right question: *who calls LevelData.Moved(), and when?*

2. Let the agent investigate — with proof demanded

Paste the Ex06 prompt: find every call site of LevelData.Moved(), explain the swap flow in Board.TickSwap, prove root cause with a play-mode eval BEFORE proposing changes.

3. The proof pair, on the projector

```
$ unity command --project-path . eval 'return "moves=" + Match3.LevelData.Instance.RemainingMove;'  
# → 20 · make a failed swap · run again → 19      ← BUG PROVEN  
# apply the one-line fix in TickSwap's revert branch  
# → 20 · failed swap · → 20                        ← FIX PROVEN (a real swap still costs 1)
```

If anything derails, switch to the recorded video without apologizing; take 5 min of questions after.

13:30 · TRACK A — EXERCISE-BY-EXERCISE RUNBOOK

Deck 04. Environment check 13:30–13:40 (per the lab guide). Every exercise starts with **everyone running ./catchup.sh exNN** — finished, behind or absent, all identical. Prompts below are abridged; the full text is in Workshop/Ex0N*.md, which participants have open.

1. Ex01 — Recon (13:40, hands-on, ~15 min)

Your demo (3 min): ask the agent to map the codebase writing nothing, then verify ONE claim in source on the projector. Their turn: the ARCHITECTURE.md prompt — scene flow, 5 key classes, match detection, booster pattern, level authoring, every claim citing a path. **Enforce: each person verifies two citations by opening the cited file.** A wrong citation is a win — make the agent correct it.

2. Ex02 — Contract (hands-on, ~15 min)

Show the stub CLAUDE.md; derive one rule live ("level tuning lives in LevelData, so data changes go through scene values — that's a contract line"). Their turn: rewrite CLAUDE.md from ARCHITECTURE.md — protected paths, an address for every kind of change, the verification ritual. The test that makes it real: **fresh agent session, ask "what are you not allowed to touch here?"** — it must answer from their contract.

3. Ex03 + Ex04 + Ex05 — your demos (14:10–14:30, ~6 min each)

Run each exercise's prompt yourself, fast, narrating the skill: **Ex03** — rebalance Level 2 via CLI (MaxMove -4 , first goal $+5$), no hand-edited YAML; verify with eval + the HUD. **Ex04** — ComboTracker subscribing to LevelData's existing

OnGoalChanged/OnMoveHappened delegates; say the review question out loud: "is it reusing hooks or inventing plumbing?"

Ex05 — Amethyst: duplicated prefab, purple tint, GemType 6, registered into Board.ExistingGems via CLI — zero C# changes; verify ExistingGems.Length == 7. Mention Track B will reskin this gem too.

4. Ex06 — BUG-312 (14:30, hands-on, ~15 min)

The morning demo, now at their keyboards on the same proof pair (20»19 buggy, 20»20 fixed). Patrol for symptom patches: any plan that raises MaxMove, clamps, or adds flags gets REJECTED — the fix names the wrong Moved() call in TickSwap's revert branch. This is the day's centerpiece; protect its time.

5. Ex07 — level_lint (14:45, hands-on, ~10 min)

Hook: unity command --project-path . | grep lint → nothing, "let's change that." Their prompt: an editor class with [CliCommand] validating the open scene (LevelData exists, goals sane, goal gems in the mix, spawners present). Then the sabotage test: set a goal's Count to 0 via eval » FAIL » undo » PASS.

6. Ex08 briefing + Ex09 demo (14:55, 5 min)

Ex08: read FEAT-88 (Cross Bomb) aloud, walk the BonusGem contract once (m_Usable, Use() » CreateCustomMatch » HandleContent), then stop — "this one you fly solo after the workshop; your prompt, your plan review, your proof; level_lint must stay green." Ex09: kick the build live and let it run into the closing —

```
$ unity command --project-path . build --target iOS --outputPath Builds/iOS --confirm true
$ unity command --project-path . build_status
```

Narrate the designed trap: the LevelList preprocessor stops the first build with "Level List had to be rebuilt" — restart once or twice; an Xcode signing error at the very end is fine.

15:15 · TRACK B — RUNBOOK

1. Flow 1: generate » clean up » swap (15:15 – 16:00)

Brief talk from deck 05 (why this game: lighting + VFX, not just sprites). Then the four steps from the lab guide, you circulating: (1) generate the 7-gem set from ONE prompt template with color variations — set consistency over single-gem beauty, prompts into the provenance record; (2) clean up and normalize to equal pixel size; (3) **copy the sample gems' import settings exactly** (PPU, pivot, filter — the classic failure is a giant/tiny gem); (4) swap on the prefabs via CLI: `SpriteRenderer.sprite` AND the Gem component's `UISprite` field (HUD icons). Anyone without Scenario access gets your pre-generated set and continues from step 2.

2. Flow 2: light » verify » approve (16:00 – 16:45)

(5) tune global + point 2D lights to the new palette — readability test: 7 gems distinguishable at a glance on the dark board; (6) map match-VFX colors through exposed VFX Graph parameters only; (7) full Level 1 run — gems read, HUD icons updated, effects match; (8) approval package: checklist + provenance + before/after screenshots, then the cross-approval rehearsal with written reasoning.

16:45 · CLOSING — RUN OF SHOW

1. Show & tell (15 min)

Track A first: one volunteer shows the BUG-312 proof pair and their level_lint sabotage test; you show the build report from the Ex09 demo. Track B: one before/after reskin on the projector. Then the merge moment — reskin + features on one branch, one more terminal build.

2. The four traps + next 90 days (deck 06, slides 3–4)

Confident misattribution · invented plumbing · symptom patching · contract-less sessions. Then the concrete steps: checkpoint discipline on real projects, level_lint » production lint in CI, an ARCHITECTURE.md + CLAUDE.md pair per active repo, quarterly metrics. **Collect the filled-in metrics templates before anyone leaves.**

WHEN THINGS BREAK

Symptom	Your move
unity status not "ready"	Editor compiling — wait. Persists: <code>unity pipeline list</code> , check for Safe Mode.
Participant hopelessly behind (Track A)	<code>./catchup.sh exNN</code> — their work is stashed, they are identical to the room.
Someone wants the solution	<code>git diff exNN-start..exNN+1-start</code> — the next branch IS the answer; copying is "reading the code review".
Agent plans outside its lane	Reject the plan in front of everyone — it's the best teaching moment of the day.
Morning Coin Rush machine broken	Close without saving, reopen the current Start scene; worst case Workshop > Build All Checkpoints.
BUG-312 demo derails	Recorded video, no apology; questions after.
Scenario down / no access	Pre-generated gem set from your machine; Track B continues from cleanup.
Editor confused after branch switch	<code>catchup.sh</code> already refreshed assets; File > Open Recent to reopen the scene.